



SEN 201: INTRODUCTION TO SOFTWARE ENGINEERING

**(Requirements Gathering &
Software Requirements
Specification (SRS))**



INSTRUCTORS: MR. AYAMBA, VICTOR

Overview

Section	Content
1	Introduction to SRS
2	Why Requirements Matter
3	Structure of an SRS Document
4	Writing Each Section
5	Common Mistakes & Best Practices
6	Practical Task (Student Activity)

INTRODUCTION

What is SRS (Software Requirements Specification)?

A Software Requirements Specification (SRS) is a formal document that describes what a software system should do, how it should behave, and any constraints or external interactions.

Purpose:

- Serves as a contract between the client and the development team.
- Provides a blueprint for design, development, and testing.
- Ensures all stakeholders have a shared understanding of the system.

Why an SRS is important

- *Prevents scope creep (adding unplanned features).*
- *Reduces miscommunication between users and developers.*
- *Provides a basis for cost and time estimation.*
- *Helps create test cases later in the project.*

SRS Standards

Software Requirements Specification (SRS) documents can follow several recognized standards that define how software requirements should be structured and presented.

1. IEEE 830 Standard

The IEEE 830 is one of the most widely used standards for writing SRS documents. It provides a clear structure including sections for introduction, overall description, specific requirements, and appendices. Its main goal is to ensure that requirements are **complete, consistent, and verifiable**, serving as a formal agreement between developers and clients.

SRS Standards

2. Volere Template

The Volere standard (developed by Suzanne and James Robertson) offers a **practical template** for requirements gathering, focusing on business goals, stakeholder needs, functional and non-functional requirements, and constraints. It is often used in agile or hybrid development settings.

3. Agile User Stories / BDD-based Specifications

In agile environments, formal SRS documents are often replaced or complemented by **user stories** and **behavior-driven specifications (BDD)**. These formats emphasize collaboration and adaptability while maintaining traceability to system requirements.

A typical SRS includes
five core sections:

Section	Description
1. Introduction	Purpose, scope, definitions
2. Overall Description	General overview of the product and constraints
3. System Features and Requirements	Functional and non-functional requirements
4. Other Requirements	Database, legal, localization, risk
5. Appendices	Supporting diagrams, glossary, TBD items

SRS in context: Crop Yield App

1. INTRODUCTION SECTION

Subsection	Description	Example (<i>CropYield Tracker</i>)
Purpose	State why this software exists	<i>The purpose of this document is to specify the requirements for the CropYield Tracker web application which helps farmers log crops and predict harvest yield.</i>
Intended Audience	Who should read this document	<i>Developers, testers, project sponsors, and agricultural consultants.</i>
Intended Use	What the system will be used for	<i>The system will be used by farmers to track and monitor crop yields and planting data.</i>
Product Scope	Boundaries and objectives	<i>A single-page offline-capable web app for yield estimation of 3 crops (tomato, maize, beans).</i>
Definitions and Acronyms	Explain key terms	<i>UI (User Interface); (FR – Functional Requirement); (NFR – Non-Functional Requirement).</i>

2. OVERALL DESCRIPTION

Subsection	Description	Example
2.1 User Needs	Explain user problems and goals	<i>Farmers need a fast, offline tool to calculate and record yields easily.</i>
2.2 Assumptions and Dependencies	Identify what the system depends on	<i>Assumes access to a smartphone and local storage; no server dependency.</i>

3. System Features and Requirements

Approaches to writing Functional Requirements

There are two approaches to writing Functional Requirements. These are:

1. EARS (Easy Approach to Requirements Syntax)
2. BDD (Behavior-Driven Development)

EARS (Easy Approach to Requirements Syntax)

EARS is a structured way of writing software requirements in clear, concise sentences.

It follows a simple pattern such as:

“When [event], the system shall [response].”

This format helps eliminate ambiguity by focusing on *what triggers a behavior* and *what the system must do in response*.

It is easy to read for both technical and non-technical audiences.

BDD (Behavior-Driven Development)

BDD, goes a step further by describing system behavior through **scenarios written in plain English**, using a format called **Gherkin syntax**:

“Given [context], when [action], then [expected outcome].”

BDD focuses on **how the system behaves from the user’s perspective** and makes requirements testable.

It bridges the gap between developers, testers, and stakeholders, ensuring everyone shares a common understanding of the system’s expected behavior.

3.1 Functional Requirements

These describe *what the system should do*.

ID	Description	Example (EARS Format)
FR-01	When a farmer selects a crop, the system shall display its average yield per m ² .	“When [user selects crop], the system shall [display average yield].”
FR-02	When a farmer enters planting date, system shall calculate harvest date.	“When [user enters planting date], the system shall [compute and show expected harvest date].”
FR-03	When the user submits data, it shall be stored locally for retrieval.	“When [data is entered], the system shall [save data in local storage].”

3.1 Functional Requirements

These describe *what the system should do using BDD and EARS FOrmat.*

ID	Description	EARS Format	BDD Format
FR-01	Display average yield	When a farmer selects a crop, the system shall display its average yield per m ² .	Scenario: Display average yield: Given the farmer is on the crop page When they select a crop Then the system displays its average yield per m ²
FR-02	Compute harvest date	When a farmer enters planting date, the system shall compute and show expected harvest date.	Scenario: Calculate harvest date Given the farmer has entered planting date When they click "Save"Then the system displays expected harvest date
FR-03	Save data locally	When the user submits data, the system shall save data in local storage.	Scenario: Save data offline Given the farmer has entered all yield data When they click "Submit"Then the system stores it locally for retrieval

3.2 Non-Functional Requirements

Define how well the system should perform.

Type	Requirement	Example
Performance	Response time	<i>95% of calculations return in under 2 seconds.</i>
Security	Access control	<i>Only registered farmers can view stored data.</i>
Usability	Interface	<i>System should be readable and usable on 320px screens.</i>
Reliability	Uptime	<i>App should run offline and not depend on internet.</i>
Compliance	Standards	<i>Must follow basic WCAG accessibility guidelines.</i>

3.3 External Interface Requirements

Defines how the system interacts with external entities.

Type	Description	Example
Performance	Hardware/software limits	<i>App should run smoothly on devices with 2GB RAM.</i>
Safety	Avoid data loss	<i>Ensure data backup during app updates.</i>
Security	Protect data	<i>Encrypt data stored locally.</i>
Software Quality Attributes	Maintainability, scalability	<i>Code should be modular and documented.</i>
Business Rules	Constraints from business	<i>Only 3 crop types supported in Phase 1.</i>

3.4 System Features

List all key features of your application.

Feature	Description
Crop Input Form	Farmers can select crop type, area, and planting date.
Yield Calculator	Displays estimated yield and harvest date instantly.
Offline Storage	Saves last 5 entries locally using browser storage.
Simple Dashboard	Allows viewing previous crop logs.

4. Other Requirements

3.4 Other Requirements

Type	Example
4.1 Database Requirements	The system will store crop data and yield logs in a local JSON file or SQLite database.
4.2 Legal & Regulatory	Must comply with open data and privacy standards (GDPR-like).
4.3 Internationalization	Support for English and French (future scope).
4.4 Risk Management	<i>(Example FMEA)</i> Risk: Incorrect yield data → Mitigation: Use validated agronomic data source.

5. Appendices

5. Appendices

Subsection	Description
5.1 Glossary	Key technical and agricultural terms.
5.2 Use Cases & Diagrams	UML Use Case and Data Flow Diagrams.
5.3 TBD (To Be Determined)	<i>Cloud backup option, user authentication (future versions).</i>

Your Task

- Create a Software Requirements Specification Document in Microsoft Word for your project idea.
- Use the template provided in class (IEEE-style format).
- Complete all the section fully week.
- Add sample Use Case Diagram or User Stories and architectures from last class.
- Submit your .docx by the next lab session.